Матрица Судеб

Искусство Цепей Маркова без решения СЛАУ

Калашников Александр

Wildberries & Russ, ФКН ВШЭ

Содержание

1 Введение: Проклятие деревьев исходов 2

2 Анатомия Цепи: Матрица Переходов 2

3 Бесконечность: Стационарное распределение 3

4 Олимпиадный Чит-код: Неориентированные графы 4

5 ШЕДЕВР: Поглощающие цепи и Матрица N 6

5.1 Разрезаем матрицу 6

5.2 Ряд Неймана (Матричная геометрическая прогрессия) 6

6 Парадокс Конвея: Матрицы "читают"тексты 8

7 Уничтожение задач уровня ВсОШ 10

8 Кем мы будем в конце? (Вероятности исходов) 12

1 Введение: Проклятие деревьев исходов

В классических задачах на случайные процессы (генерация текстов LLM, блуждания по графам, игры с разорением) школьники обычно используют Метод первых шагов. Они вводят ожидаемое время E_i для каждого состояния, составляют Систему Линейных Алгебраических Уравнений (СЛАУ) и пытаются решить её вручную.

Если состояний 3, это терпимо. Но если их 5, 10 или 50? Решать СЛАУ методом Гаусса на бумаге — это гарантированная ошибка в арифметике. Более того, возня с дробями скрывает от нас внутреннюю геометрию процесса.

Сегодня мы переведём случайную величину на язык высшей алгебры. Мы узнаем, как предсказывать будущее системы на миллион шагов вперёд одним умножением, и как знаменитая школьная формула бесконечной убывающей геометрической прогрессии $S = \frac{1}{1-q}$ элегантно обобщается на многомерные пространства.

2 Анатомия Цепи: Матрица Переходов

Определение 2.1: Цепь Маркова

Случайный процесс называется Цепью Маркова, если он обладает отсутствием памяти (Марковским свойством): вероятность перехода в следующее состояние зависит только от текущего и не зависит от истории того, как система в него пришла.

Зададим n состояний. Пусть p_{ij} — вероятность того, что система из состояния i за ровно один шаг перейдёт в состояние j. Упакуем все эти вероятности в квадратную матрицу P размера $n \times n$.

Олимпиадный инсайт: Свойство Матрицы P (Стохастичность)

Сумма элементов в каждой строке матрицы P обязана быть строго равна 1. Почему? Строка i описывает все возможные пути из состояния i. Так как система обязана куда-то перейти (или хотя бы остаться на месте с вероятностью p_{ii}), сумма шансов образует полную группу событий, то есть 1.

Обозначим через вектор-строку $\boldsymbol{\pi}_0 = (p_1, p_2, \ldots, p_n)$ наше начальное распределение (с какими вероятностями мы находимся в состояниях в момент времени 0). Распределение системы через 1 шаг находится матричным умножением: $\boldsymbol{\pi}_1 = \boldsymbol{\pi}_0 P$.

Теорема 2.1: Матрица Будущего

Распределение вероятностей состояний ровно через k шагов равно:

$$\mathbf{ß_k} = \mathbf{ß_0 P^k}$$

В олимпиадном программировании это позволяет находить точные вероятности для $k = 10^{18}$ за $\mathcal{O}(\log k)$ с помощью алгоритма бинарного возведения матрицы в степень!

3 Бесконечность: Стационарное распределение

Представьте, что бот бесконечно блуждает по ссылкам в интернете. Куда он придёт через $k \to \infty$ шагов? Теорема Перрона — Фробениуса гласит: если граф сильно связен (из любой вершины можно дойти до любой) и апериодичен, система рано или поздно забудет своё начальное состояние $\boldsymbol{\pi}_0$ и сойдётся к единому глобальному равновесию.

Это равновесие называется Стационарным распределением $\boldsymbol{\pi}$. Раз оно равновесное, то при применении к нему ещё одного шага (умножения на матрицу P), оно не должно измениться! Получаем элегантное уравнение:

$$\mathbf{ß}\mathbf{P} = \mathbf{ß}$$

Внимание: Собственные векторы и Спектральная Щель

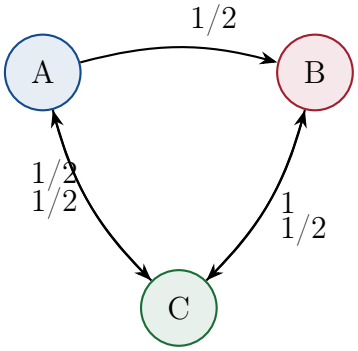
Посмотрите на уравнение $\boldsymbol{\pi} P = 1 \cdot \boldsymbol{\pi}$. Это в точности определение левого собственного вектора матрицы P, соответствующего собственному значению $\lambda_1 = 1$. Линейная алгебра гарантирует, что у любой стохастической матрицы такой вектор всегда существует.

А как быстро система забудет старт? Это зависит от второго по модулю собственного значения $|\lambda_2| < 1$. Величина $\gamma = 1 - |\lambda_2|$ называется спектральной щелью (Spectral Gap). Чем она больше, тем быстрее граф «перемешивает» вероятности. На этом свойстве построены алгоритмы MCMC и экспандерные графы.

Задача 3.1: PageRank: Самая дорогая матрица в истории

Интернет состоит из 3 страниц: A, B, C. Из A есть ссылки на B и C. Из B ссылка только на C. Из C ссылки на A и B. Пользователь кликает по случайной ссылке на текущей странице. Какую долю времени он будет проводить на странице C в пределе?

РЕШЕНИЕ. Алгоритм Google PageRank — это буквально поиск стационарного вектора $\boldsymbol{\pi}$ матрицы интернета! Составим граф и матрицу P (строки A, B, C):



$$P = \begin{pmatrix} 0 & 1/2 & 1/2 \\ 0 & 0 & 1 \\ 1/2 & 1/2 & 0 \end{pmatrix}$$

Решаем систему $\boldsymbol{\pi} P = \boldsymbol{\pi}$ (умножая строку на столбцы): $\begin{cases} \pi_A = \frac{1}{2}\pi_C \\ \pi_B = \frac{1}{2}\pi_A + \frac{1}{2}\pi_C \\ \pi_C = \frac{1}{2}\pi_A + 1\pi_B \end{cases}$

Из первого: $\pi_C = 2\pi_A$. Подставляем во второе: $\pi_B = 0.5\pi_A + 0.5(2\pi_A) = 1.5\pi_A$. Помним условие нормировки вероятностей: $\pi_A + \pi_B + \pi_C = 1$. Тогда $\pi_A + 1.5\pi_A + 2\pi_A = 1 \implies 4.5\pi_A = 1 \implies \pi_A = 2/9$.

Вектор $\boldsymbol{\pi} = \left(\frac{2}{9}, \frac{3}{9}, \frac{4}{9}\right)$. Пользователь проводит $\approx 44\%$ времени на странице C. ■

4 Олимпиадный Чит-код: Неориентированные графы

Решать СЛАУ $\boldsymbol{\pi} P = \boldsymbol{\pi}$ для графа на 100 вершин нереально. Но олимпиадники обожают давать задачи на блуждания по неориентированным графам (где дороги двусторонние). Здесь кроется гениальный математический шорткат.

Олимпиадный инсайт: Принцип Детального Равновесия

В физике и теории графов существует понятие обратимости. Если граф неориентированный, поток вероятности из вершины u в v в стационаре в точности равен потоку из v в u: $\pi_u p_{uv} = \pi_v p_{vu}$. Так как шаг делается случайно, $p_{uv} = \frac{1}{\deg(u)}$. Подставляя это, получаем $\frac{\pi_u}{\deg(u)} = \frac{\pi_v}{\deg(v)} = \text{const}$. Это означает, что вероятности прямо пропорциональны степеням вершин!

Теорема 4.1: Стационарность неориентированных графов

В случайном блуждании по связному неориентированному графу доля времени, проведённая в вершине v, прямо пропорциональна её степени $\deg(v)$:

$$\mathbf{ß_v} = \frac{\deg(\mathbf{v})}{\sum \deg(\mathbf{u})} = \frac{\deg(v)}{2|E|}$$

А ожидаемое время возвращения (через сколько шагов бот впервые вернётся в v, если стартовал из v) равно обратной величине: $\mathbb{E}[\mathbf{ø_{v \to v}}] = \frac{\mathbf{1}}{\mathbf{ß_v}}$.

Задача 4.1: Прогулка Короля (Эргодическая цепь)

Шахматный король бесконечно долго делает случайные разрешенные ходы по пустой доске 8×8. 1. Какую долю времени он проводит в 4 центральных клетках (d4, d5, e4, e5)?
2. Если король стоит в углу (a1), через сколько ходов он в среднем вернётся в a1?

РЕШЕНИЕ. Доска — это граф из 64 вершин. Степень — количество возможных ходов.

- 4 угла: степень 3. Сумма $= 4 \times 3 = 12$.
- 24 клетки на краях (без углов): степень 5. Сумма $= 24 \times 5 = 120$.
- 36 внутренних клеток: степень 8. Сумма $= 36 \times 8 = 288$.

Общая сумма степеней (удвоенное число рёбер графа) $\sum \deg = 12 + 120 + 288 = \mathbf{420}$.
1) Центральных клеток 4, их суммарная степень равна $4 \times 8 = \mathbf{32}$. Ответ: Король находится в центре $\frac{\mathbf{32}}{\mathbf{420}} = \frac{\mathbf{8}}{\mathbf{105}} \approx 7.62\%$ времени.
2) Степень угла a1 равна 3. Стационарная вероятность $\pi_{a1} = \frac{3}{420} = \frac{1}{140}$. Время возврата равно перевернутой дроби: $\mathbb{E}[\tau_{a1 \to a1}] = \mathbf{140}$ ходов. ■

Внимание: Ловушка Четности: Конь против Короля

А сработала бы формула $32/420$ для Коня? И да, и нет. Конь при каждом ходе меняет цвет клетки. Граф ходов коня — двудольный.
Из-за этого строгий математический предел $\lim_{n \to \infty} \boldsymbol{\pi}_0 P^n$ не существует (вероятности осциллируют: на четных шагах конь 100% на белых клетках, на нечетных 100% на

черных). Однако, усредненная доля времени (предел по Чезаро) и время возврата всё равно идеально подчиняются формуле! Жюри ВсОШ обожает снимать баллы за отсутствие оговорки про двудольность.

5 ШЕДЕВР: Поглощающие цепи и Матрица N

Стационарное распределение ищет бесконечный предел. Но в олимпиадах игра чаще всего заканчивается (муха долетела до паука, игрок разорился, сгенерировалась нужная строка). Такие финальные состояния называются поглощающими ($p_{ii} = 1$).

Для решения таких задач существует феноменальная Блочная Теорема.

5.1 Разрезаем матрицу

Запишем матрицу P в каноничном блочном виде. Сначала идут t «транзитных» (временных) состояний, затем r «поглощающих» (финишей).

$$P = \left(\begin{array}{c|c} Q & R \\ \hline 0 & I \end{array}\right)$$

Где Q — квадратная матрица $t \times t$, описывающая переходы строго между временными состояниями, а R — вероятности прыжка в финиш.

5.2 Ряд Неймана (Матричная геометрическая прогрессия)

Посчитаем ожидаемое количество визитов в каждое временное состояние. Благодаря линейности матожидания, ожидаемое число визитов — это сумма вероятностей оказаться там на каждом шаге $k = 0, 1, 2 \dots$

В момент 0 мы находимся в старте (единичная матрица I). На шаге 1 нас описывает матрица Q. На шаге 2 — матрица Q^2. Полное ожидаемое число посещений — это сумма бесконечного матричного ряда: $N = I + Q + Q^2 + Q^3 + \dots$

Вспомним школьную формулу суммы убывающей геометрической прогрессии: $1 + q + q^2 + \dots = \frac{1}{1-q} = (1-q)^{-1}$. Работает ли это для матриц? ДА! Доказательство:

$$(I - Q)(I + Q + Q^2 + \dots) = I + Q + Q^2 \dots - Q - Q^2 - Q^3 \dots = I$$

Умножив на обратную матрицу, получаем $(I - Q)^{-1} = I + Q + Q^2 + \dots$

Теорема 5.1: Фундаментальная Матрица (Матрица Времени)

Ожидаемое количество посещений каждого состояния до поглощения задаётся Фундаментальной матрицей N:

$$\mathbf{N} = (\mathbf{I} - \mathbf{Q})^{-1}$$

Элемент N_{ij} показывает, сколько раз в среднем мы посетим состояние j, если начали в i.
Чтобы найти общее ожидаемое время игры E_i из состояния i, нужно просто сложить все элементы i-й строки: $\tilde{\mathbf{E}} = \mathbf{N} \cdot \mathbf{1}$ (где $\mathbf{1}$ — вектор из единиц).

Олимпиадный инсайт: Инверсия $(I - Q)$ — это и есть Метод Первых Шагов!

Если вы запишете систему уравнений времени $\vec{E}$ для графа, вы получите: $\vec{E} = \mathbf{1} + Q\vec{E}$. Перенесем $\vec{E}$ влево: $\vec{E} - Q\vec{E} = \mathbf{1} \implies (I - Q)\vec{E} = \mathbf{1}$. Умножив слева на обратную матрицу, получим $\vec{E} = (I - Q)^{-1}\mathbf{1}$. Вся "магия"марковских цепей — это просто

элегантная матричная запись решения классической СЛАУ!

6 Парадокс Конвея: Матрицы "читают"тексты

В методичке по Мартингалам мы обсуждали знаменитый парадокс генерации строк. Монета бросается до выпадения слов $W_1 = \text{HT}$ (Орёл-Решка) или $W_2 = \text{HH}$ (Орёл-Орёл). Одинаково ли среднее время ожидания?

Давайте посмотрим, как Фундаментальная матрица расправляется с этой задачей без всяких искусственных инвариантов. Транзитные состояния автомата — это наибольший префикс искомого слова, который мы уже набрали.

Задача 6.1: Ожидание строки HT

Генерируем H и T с вероятностью $1/2$. Ждем появление HT. Транзитные состояния: $\varnothing$ (пусто), H. Поглощающее: HT.

РЕШЕНИЕ. Составим матрицу Q (строки/столбцы $\varnothing$, H):

- Из $\varnothing$: при T остаемся в $\varnothing$ $(1/2)$, при H идем в H $(1/2)$.
- Из H: при H топчемся на месте в H $(1/2)$, при T уходим в поглощение (исчезаем из Q).

$$Q = \begin{pmatrix} 1/2 & 1/2 \\ 0 & 1/2 \end{pmatrix} \implies I - Q = \begin{pmatrix} 1/2 & -1/2 \\ 0 & 1/2 \end{pmatrix}$$

Определитель $\det(I - Q) = 1/4$. Обращаем матрицу 2×2:

$$N = (I - Q)^{-1} = 4 \begin{pmatrix} 1/2 & 1/2 \\ 0 & 1/2 \end{pmatrix} = \begin{pmatrix} 2 & 2 \\ 0 & 2 \end{pmatrix}$$

Время из старта $\varnothing$ (сумма 1-й строки): $2 + 2 = \mathbf{4}$ броска. ■

Задача 6.2: Ожидание строки HH

Транзитные состояния: $\varnothing$ (пусто), H. Поглощающее: HH.

РЕШЕНИЕ. Матрица Q (строки/столбцы $\varnothing$, H):

- Из $\varnothing$: при T остаемся в $\varnothing$ $(1/2)$, при H идем в H $(1/2)$.
- Из H: при T сбрасываемся в $\varnothing$ $(1/2)$, при H уходим в поглощение.

$$Q = \begin{pmatrix} 1/2 & 1/2 \\ 1/2 & 0 \end{pmatrix} \implies I - Q = \begin{pmatrix} 1/2 & -1/2 \\ -1/2 & 1 \end{pmatrix}$$

Определитель $\det = 1/2 - 1/4 = 1/4$. Обращаем матрицу:

$$N = (I - Q)^{-1} = 4 \begin{pmatrix} 1 & 1/2 \\ 1/2 & 1/2 \end{pmatrix} = \begin{pmatrix} 4 & 2 \\ 2 & 2 \end{pmatrix}$$

Время из старта $\varnothing$ (сумма 1-й строки): $4 + 2 = \mathbf{6}$ бросков. ■

Олимпиадный инсайт: Матрица "видит"структуру слова

Посмотрите на элемент N_{11} (ожидаемое количество возвратов в $\varnothing$). Для HT оно равно 2, а для HH оно равно 4! Матрица автоматически учла тот факт, что неудач-

ный бросок (Т) при ожидании НН отбрасывает нас в самое начало, тогда как при ожидании НТ мы можем безопасно "топтаться"на букве Н (ведь ТН заканчивается на Н). Фундаментальная матрица N содержит в себе всю теорию детерминированных автоматов и префикс-функций в чистом алгебраическом виде!

7 Уничтожение задач уровня ВсОШ

Применим матричное оружие к зубодробительной задаче с регионального этапа. Мы не просто решим её, но и вскроем комбинаторику переходов, которая обычно ставит всех в тупик.

Задача 7.1: Три мухи на пятиугольнике (ВсОШ)

Три мухи одновременно садятся на правильный пятиугольник — в 3 различные вершины. Каждую минуту каждая муха случайно и независимо перелетает в одну из 4 других вершин (отличных от текущей). Найдите ожидаемое время $\mathbb{E}[T]$, когда все три мухи впервые окажутся в одной вершине.

РЕШЕНИЕ. У трёх мух $4^3 = 64$ возможных варианта прыжка. Разобьем состояния на 3 макросостояния:

- S3: Все мухи в разных вершинах (старт).
- S2: Две мухи в одной вершине, одна отдельно.
- S1: Все мухи в одной вершине (Поглощающее состояние).

Шаг 1. Вскрытие комбинаторики (Из S3). Пусть мухи сидят в A, B, C.

- В S1: Они должны прыгнуть в одну общую вершину. Прыгать в A, B, C всем вместе нельзя (кто-то нарушит правило и останется на месте). Значит, они летят либо все в D, либо все в E. $\implies$ $\mathbf{p_{31} = 2/64}$.
- В S3: Все 3 цели (v_1, v_2, v_3) должны быть различны. Всего способов выбрать 3 различные цели из 5: $5 \times 4 \times 3 = 60$. Но нам нужно вычесть те, где кто-то остался на месте. Применим Формулу Включений-Исключений (PIE). Свойства P_1, P_2, P_3 — муха осталась на месте. Один на месте: если $v_1 = A$, две другие мухи выбирают 2 различные цели из оставшихся 4 вершин $\implies$ $4 \times 3 = 12$. Таких свойств три $\implies$ $\Sigma = 36$. Двое на месте: если $v_1 = A, v_2 = B$, третья муха выбирает из 3 вершин $\implies$ 3. Таких пар три $\implies$ $\Sigma = 9$. Трое на месте: $v_1 = A, v_2 = B, v_3 = C$ $\implies$ 1. Число валидных исходов: $60 - 36 + 9 - 1 = \mathbf{32}$. $\implies$ $\mathbf{p_{33} = 32/64}$.
- В S2 (остаток): $64 - 2 - 32 = 30$ способов. $\implies$ $\mathbf{p_{32} = 30/64}$.

Шаг 2. Вскрытие комбинаторики (Из S2). Пусть мухи сидят в A, A, B.

- В S1: Прыгают все вместе в свободные C, D или E (3 способа). $\implies$ $\mathbf{p_{21} = 3/64}$.
- В S3: Снова ищем (v_1, v_2, v_3) различные (60 вариантов). Вычитаем нарушения $(v_1 = A \vee v_2 = A \vee v_3 = B)$. Один на месте: $|P_1| = 12, |P_2| = 12, |P_3| = 12$. Сумма: 36. Двое на месте: $|P_1 \cap P_2| = 0$ (цели обязаны быть различны, они не могут обе прыгнуть в A!). $|P_1 \cap P_3| = 1 \times 3 \times 1 = 3$. $|P_2 \cap P_3| = 3$. Сумма пар: $0 + 3 + 3 = 6$. Трое на месте $\implies$ 0. Число валидных исходов: $60 - 36 + 6 - 0 = \mathbf{30}$. $\implies$ $\mathbf{p_{23} = 30/64}$.
- В S2 (остаток): $64 - 3 - 30 = 31$ способ. $\implies$ $\mathbf{p_{22} = 31/64}$.

Шаг 3. Фундаментальная Матрица. Выписываем матрицу Q (индексы S3, S2) и вычитаем из I:

$$I - Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \frac{1}{64}\begin{pmatrix} 32 & 30 \\ 30 & 31 \end{pmatrix} = \frac{1}{64}\underbrace{\begin{pmatrix} 32 & -30 \\ -30 & 33 \end{pmatrix}}_{M}$$

Для обращения матрицы воспользуемся линейным свойством $(kM)^{-1} = \frac{1}{k}M^{-1}$. Значит $(I - Q)^{-1} = 64 \cdot M^{-1}$. Формула для 2×2: $M^{-1} = \frac{1}{\det M}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$. Определитель

$\det(M) = 32 \cdot 33 - (-30)(-30) = 1056 - 900 = \mathbf{156}$.
Тогда Фундаментальная матрица N:

$$N = 64 \cdot \frac{1}{156} \begin{pmatrix} 33 & 30 \\ 30 & 32 \end{pmatrix} = \frac{\mathbf{16}}{\mathbf{39}} \begin{pmatrix} 33 & 30 \\ 30 & 32 \end{pmatrix}$$

Нам нужно ожидаемое время старта из состояния S3 (первая строка матрицы N). Просто суммируем элементы первой строки:

$$\mathbb{E}[T] = \frac{16}{39}(33 + 30) = \frac{16 \cdot 63}{39} = \frac{16 \cdot 21}{13} = \frac{\mathbf{336}}{\mathbf{13}}$$

Без громоздких систем уравнений и подстановок мы получили идеальный ответ $336/13 \approx 25.85$ минут! ■

8 Кем мы будем в конце? (Вероятности исходов)

Иногда нас интересует не время игры, а исход. Если поглощающих состояний несколько (например, «Игрок победил» или «Разорился»), как узнать вероятность закончить в каждом из них?

Теорема 8.1: Матрица Итога

Вероятность того, что стартовав в транзитном состоянии i, система погибнет в конкретном поглощающем состоянии j, задаётся матрицей B:

$$\mathbf{B} = \mathbf{N} \cdot \mathbf{R} = (\mathbf{I} - \mathbf{Q})^{-1}\mathbf{R}$$

Здесь N описывает, сколько раз мы побываем на «трамплинах» перед финалом, а R — вероятности самого финального смертельного прыжка с этих трамплинов в цель.

Задача 8.1: Игра с разорением (Связь с Мартингалами)

У игрока 1 рубль. Он выигрывает 1 рубль с вероятностью p и проигрывает с q. Игра заканчивается, когда у него 0 (Разорение) или 3 рубля (Победа). Найти вероятность победы матричным методом.

РЕШЕНИЕ. Транзитные состояния: 1, 2. Поглощающие: 0, 3. Матрица $Q = \begin{pmatrix} 0 & p \\ q & 0 \end{pmatrix} \implies I - Q = \begin{pmatrix} 1 & -p \\ -q & 1 \end{pmatrix}$. Определитель $\det(I - Q) = 1 - pq$. Фундаментальная матрица: $N = \frac{1}{1-pq}\begin{pmatrix} 1 & p \\ q & 1 \end{pmatrix}$.

Матрица прыжков в финал R (строки: из 1 и 2; столбцы: в 0 и 3). Из 1 в 0 шанс q, в 3 шанс 0. Из 2 в 0 шанс 0, в 3 шанс p. $R = \begin{pmatrix} q & 0 \\ 0 & p \end{pmatrix}$.

Матрица исходов $B = N \cdot R$:

$$B = \frac{1}{1-pq}\begin{pmatrix} 1 & p \\ q & 1 \end{pmatrix}\begin{pmatrix} q & 0 \\ 0 & p \end{pmatrix} = \frac{1}{1-pq}\begin{pmatrix} q & \mathbf{p^2} \\ q^2 & p \end{pmatrix}$$

Смотрим первую строку (старт из 1 рубля) и второй столбец (состояние 3 — Победа): вероятность равна $\frac{\mathbf{p^2}}{\mathbf{1-pq}}$.

Абсолютный триумф алгебры: В методичке по Мартингалам мы выводили формулу победы $\frac{(q/p)-1}{(q/p)^3-1}$. Распишем разность кубов: $\frac{r-1}{r^3-1} = \frac{1}{r^2+r+1}$. Подставим $r = q/p$:

$$\frac{1}{(q/p)^2 + q/p + 1} = \frac{p^2}{q^2 + pq + p^2}$$

Заметим, что так как $p + q = 1 \implies (p+q)^2 = 1 \implies p^2 + 2pq + q^2 = 1 \implies q^2 + pq + p^2 = 1 - pq$. Подставляем это в знаменатель и получаем ровно $\frac{\mathbf{p^2}}{\mathbf{1-pq}}$. Матрицы и Мартингалы дали абсолютно идентичный результат. Цикл замкнулся! ■

Почему это важно для ИИ? (Reinforcement Learning)

В классических Цепях Маркова вероятности P фиксированы законами физики или правилами игры. Но что если мы дадим агенту (например, нейросети) право выбора действий a в каждом состоянии?
Тогда матрица P превращается в функцию от действий: $P(a)$. Процесс становится Марковским Процессом Принятия Решений (MDP). Вся область Reinforcement Learning (включая алгоритм RLHF, который сделал ChatGPT умным) — это алгоритмы поиска такой матрицы действий $P(a)$ (называемой Policy), которая максимизирует итоговую награду $V = (I - \gamma P)^{-1}R$, где γ — discount factor. Знаменитое Уравнение Беллмана — это в точности наша матрица N!

/ Шпаргалка ML-Разработчика: Цепи Маркова в NumPy

В олимпиадах по ИИ (и в промышленном Data Science) матрицу $(I - Q)^{-1}$ руками не считают. Python решает эту задачу в 4 строки, даже если у вас 1000 состояний.

```
import numpy as np
# 1. Поиск ожидаемого времени (Поглощающая цепь)
Q = np.array([[32/64, 30/64], [30/64, 31/64]])
# Используем linalg.solve (численно стабильнее, чем inv)
times = np.linalg.solve(np.eye(len(Q)) - Q, np.ones(len(Q)))
print(times[0]) # Ответ: 25.84615...

# 2. Поиск Стационарного распределения (Эргодическая цепь)
P = np.array([[0, 0.5, 0.5], [0, 0, 1], [0.5, 0.5, 0]])
eigenvalues, eigenvectors = np.linalg.eig(P.T) # Левый вектор = P.T
pi = eigenvectors[:, np.isclose(eigenvalues, 1)]
pi = pi / np.sum(pi) # Нормировка вектора в сумму 1
print(pi.real.flatten()) # Ответ: [0.222, 0.333, 0.444]
```

Этот код работает за $\mathcal{O}(V^3)$ и легко вскрывает NLP задачи на генерацию токенов, где классическое ДП убило бы вас сложностью реализации.